UNIVERSIDADE FEDERAL DO CEARÁ
DEPARTAMENTO DE COMPUTAÇÃO

Relatório Trabalho Final Tópicos Avançados em Aprendizagem de Máquina (CK0475)

Professor:

Eliezer de Souza da Silva

Alunos:

Davi Teixeira Silva - 584250
Guilherme Galvão Serra Silva - 520402
Lucas Rodrigues Aragão - 538390

Janeiro - 2026

1. Resumo

Neste trabalho tivemos como objetivo o estudo da arquitetura *Transformer*, discutindo as principais contribuições do artigo base "*Attention is all you need*". Além disso, fizemos a implementação da arquitetura em *python* e testando ela para o problema de autômatos celulares, fazendo uma comparação experimental com uma rede LSTM, analisando o comportamento dos modelos por meio das métricas de acurácia, *Hamming Loss* e F1-*score*.

2. Contextualização

Atualmente é possível observar o crescimento de modelos e sistemas que utilizam as chamadas LLMs (Large Language Models), como GPT, Gemini e Grok. O surgimento desses modelos se deve, entre outros fatores, pela arquitetura transformer introduzida em 2017 por pesquisadores do Google no artigo "*Attention is All You Need*", que na época, estudavam como melhorar os seus *softwares* de tradução de texto.

O surgimento do *transformer* permitiu não somente o surgimento de novas tecnologias, como as citadas acima, mas também a revisão de modelos e problemas já existentes, tentando aplicá-los para obter resultados melhores.

Por isso, escolhemos como tema do nosso trabalho final da disciplina a exploração da arquitetura transformer, fazendo uma implementação simples e validando ela para o problema de autômatos celulares, comparando com um modelo LSTM.

3. Contribuições

O principal conjunto de contribuições deste trabalho pode ser organizado em três dimensões: conceitual, metodológica e experimental.

Do ponto de vista **conceitual**, o trabalho contribui para a compreensão do papel do mecanismo de self-attention como alternativa à recorrência na modelagem de sequências. A partir da leitura aprofundada do artigo "Attention Is All You Need", discutimos como o *Transformer* elimina a dependência sequencial imposta por arquiteturas recorrentes e passa a capturar dependências de longo alcance por meio de atenção global e paralelizável. Essa análise é contextualizada historicamente por meio da comparação com modelos recorrentes clássicos, como LSTMs, evidenciando as limitações estruturais que motivaram o surgimento do *Transformer*.

Sob o aspecto **metodológico**, o trabalho propõe o uso de autômatos celulares elementares como uma tarefa sintética, controlada e determinística para validação de arquiteturas baseadas em *attention*. Diferentemente de problemas tradicionais de NLP, esse tipo de *dataset* permite controlar explicitamente a complexidade temporal do problema por meio do número de passos evolutivos (K), além de evitar custos computacionais elevados e ambiguidades inerentes a dados naturais. Essa escolha possibilita uma análise mais clara

do comportamento dos modelos e de sua capacidade de capturar interações de longo alcance.

Do ponto de vista **experimental**, o trabalho apresenta uma implementação completa e funcional de um *Transformer Encoder–Decoder* em *PyTorch*, adaptado para modelagem de sequências binárias, bem como uma implementação de uma arquitetura LSTM utilizada como *baseline*. A comparação empírica entre os modelos, em diferentes níveis de complexidade temporal, evidencia vantagens do mecanismo de *attention* em cenários de dependências não-locais, além de expor limitações comuns a ambos os modelos em regimes mais complexos (K elevados). A análise das métricas e das curvas de treinamento contribui para uma compreensão prática das capacidades e limitações do *Transformer* em tarefas além do NLP tradicional. Os experimentos foram conduzidos com foco em entendimento comportamental dos modelos, e não em maximização de desempenho absoluto, reforçando o caráter didático e exploratório do estudo.

4. Implementação

Para validar a eficácia da arquitetura *Transformer* em tarefas de modelagem de sequências complexas, implementamos um ambiente experimental comparativo utilizando o problema dos Autômatos Celulares Elementares (ECA). A implementação foi dividida em três etapas: geração do *dataset* sintético, construção da arquitetura *Transformer* (*Encoder-Decoder*) e construção da baseline com LSTM.

Diferente de tarefas de tradução de linguagem natural (NLP), optamos por um problema determinístico, mas não-linear: a evolução de um autômato celular. Especificamente, utilizamos a Regra 110, conhecida por ser *Turing*-completa e apresentar comportamento complexo na fronteira do caos.

A geração dos dados foi implementada da seguinte forma:

1. Regra de Transição: Foi definido um dicionário RULE_110 que mapeia a vizinhança de 3 células (ti−1,ti,ti+1) para o estado da célula no próximo passo de tempo (t+1).
2. Evolução Temporal (K): Implementamos uma função evolve(state, K) que aplica a regra de transição iterativamente K vezes sobre um estado inicial. O parâmetro K define a "distância" temporal que o modelo precisa aprender a prever.
3. *Dataset*: A classe CellularAutomataDataset gera vetores binários aleatórios de tamanho fixo (seq_len=64) como estados iniciais (Input) e calcula seus estados finais após K passos (Target). Isso formula o problema como uma tarefa de sequence-to-sequence supervisionada.

Implementamos uma arquitetura Transformer completa ("*Attention Is All You Need*"), composta por um Encoder e um Decoder, adaptada para processar sequências binárias discretas. A implementação no PyTorch seguiu os seguintes componentes:

- *Embeddings*: Utilizamos camadas de `nn.Embedding` para converter os *tokens* discretos (0 e 1) em vetores densos de dimensão n_embd=128, somados a embeddings posicionais (enc_pos, dec_pos) para fornecer noção de ordem à rede.

- *Encoder*: Responsável por processar o estado inicial do autômato. É composto por blocos (*EncoderBlock*) contendo camadas de *Multi-Head Self-Attention* (sem máscara, permitindo atenção bidirecional) seguidas por redes *FeedForward*, com normalização (*LayerNorm*) e conexões residuais.
- *Decoder*: Responsável por gerar o estado final. Seus blocos (*DecoderBlock*) possuem uma característica distinta fundamental:
    1. Masked Self-Attention: Garante a propriedade auto-regressiva, impedindo que o modelo "veja" tokens futuros da sequência alvo.
    2. Cross-Attention: Mecanismo onde as *Queries* (Q) vêm do Decoder e as *Keys* (K) e *Values* (V) vêm da saída do Encoder. Isso permite que o modelo foque em partes específicas do estado inicial para determinar o estado final.

Para fins de comparação, implementamos uma rede neural recorrente baseada em LSTM (*Long Short-Term Memory*). A arquitetura LSTMCellularAutomata consiste em:

- Uma camada de *Embedding* idêntica à do *Transformer*.
- Uma camada nn.LSTM com hidden_dim=128 e 2 camadas empilhadas, processando a sequência passo a passo.
- Uma camada linear (fc) na saída para projetar os estados ocultos de volta ao espaço de vocabulário (bits 0 ou 1).

A LSTM serve como baseline para verificar se o mecanismo de atenção global do *Transformer* oferece vantagem sobre a memória sequencial recorrente ao modelar as interações de longo alcance geradas pela evolução do autômato.

Ambos os modelos foram treinados utilizando o otimizador AdamW com taxa de aprendizado de 3×10^{-4}. A função de perda utilizada foi a *Cross Entropy Loss*, tratando cada posição da fita do autômato como uma classificação binária. O treinamento incluiu um mecanismo de *Early Stopping* baseado na convergência da loss de treinamento para evitar processamento desnecessário em tarefas mais simples (como K=1). As métricas de avaliação incluíram Acurácia, Hamming Loss e F1-Score.

Durante o treinamento do *Transformer*, foi utilizado *teacher forcing*, no qual a sequência alvo correta é fornecida ao decoder como entrada. Essa escolha visa isolar a capacidade do modelo de aprender a função de transição do autômato celular, sem introduzir instabilidades associadas à geração auto-regressiva livre.

5. Experimentos

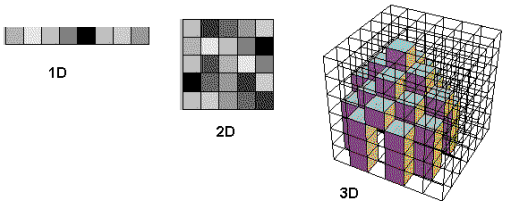
5.1. Datasets

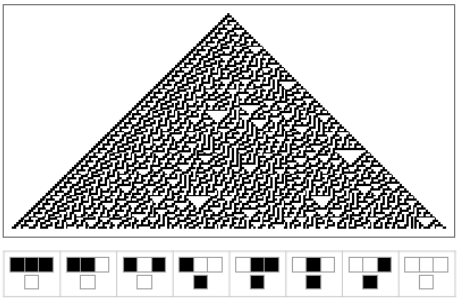
A ideia inicial que tivemos foi de tentar replicar os experimentos do artigo original utilizando datasets de tradução. Para isso, pegamos o conjunto de tradução de alemão para inglês, que possui mais de 4 milhões de instâncias. Fizemos um sampling de 10% do conjunto de treinamento para tentar conseguir rodar localmente. Entretanto, mesmo com a

redução, ao tentar rodar, percebemos que o tempo necessário para o treinamento seria de aproximadamente 3 semanas, obviamente algo inviável.

Com isso, a alternativa encontrada foi mudar de problema. Onde encontramos o problema dos autômatos celulares.

Os autômatos celulares podem ser definidos como "*Modelos computacionais discretos compostos por uma grade de células, onde cada célula assume um estado finito que evolui em passos de tempos discretos.*". O valor de cada célula normalmente provém de uma regra de vizinhança "simples", mas que possuem a capacidade de gerar padrões gerais complexos e difíceis de se aprender, sendo necessário entender o **contexto** para aferir o valor correto.





Apesar de parecerem simples à primeira vista, o problema se mostrou capaz de levar o transformer implementado a regimes de maior dificuldade nos experimentos.

Para criação do dataset de treinamento usamos uma das regras de criação mais famosas para autômatos celulares, a **regra 110**. Para esse caso, assumimos um autômato celular de 1 dimensão e binário, ou seja, cada célula assume valor de 0 ou 1. A regra se baseia na vizinhança de uma célula para definir seu valor no próximo passo, seguindo a tabela abaixo:

Padrão	111	110	101	100	011	010	001	000
Valor da célula central	0	1	1	1	0	1	1	0

Tabela com os padrões da regra usada para criação do dataset.

Para a criação do dataset usado nos experimentos, criamos 10000 autômatos randômicos para treino e 2000 para testes, ambos com 64 células cada um. Após isso, damos K passos à frente, ou seja, K passos "evolutivos" na sequência, usando a regra de criação. Salvamos no dataset o valor do vetor inicial e o valor do vetor após os passos.

5.2. Metodologia dos experimentos

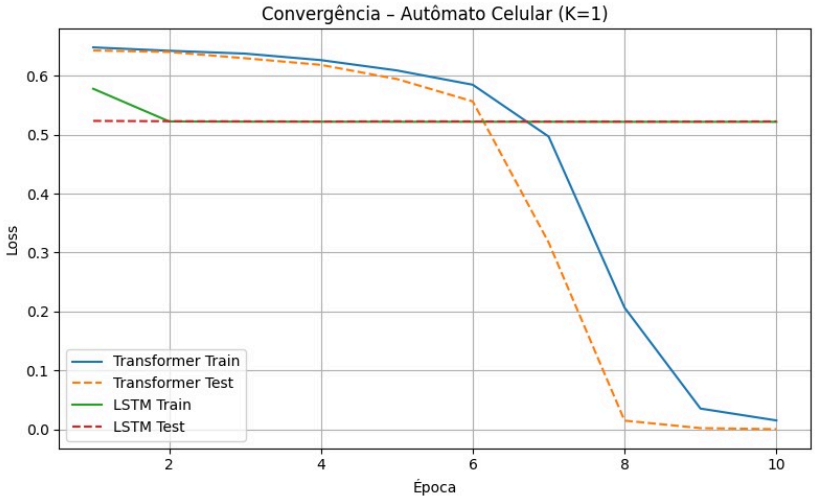
Para comparar a qualidade da implementação do transformer, criamos um modelo recorrente com camadas LSTM, usando a implementação da biblioteca *Pytorch*. Para

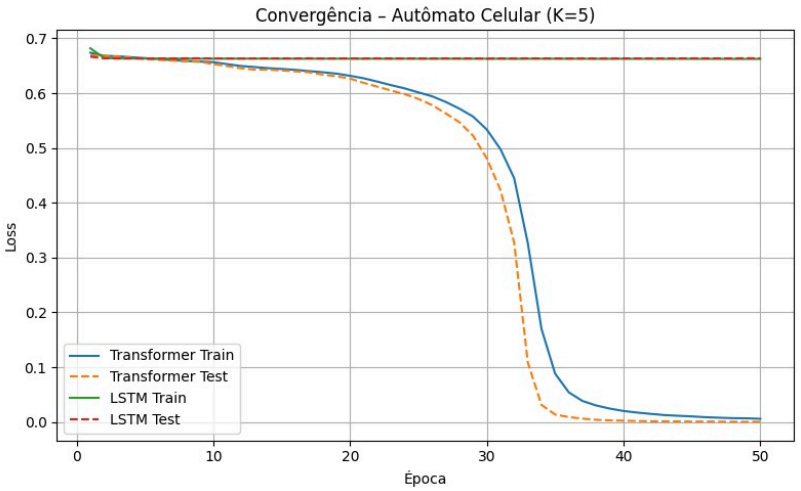
observar o aprendizado do modelo usamos a *cross entropy loss* como função de perda para ambos os modelos.

Para observar a qualidade das predições dos modelos, usamos 3 métricas clássicas na literatura e que se mostraram compatíveis com o problema de classificação binária. São elas, a acurácia, a hamming loss e o F1 score. A ideia geral foi variar o valor de K para os valores de K = 1, K = 5 e K = 20, e observar se o comportamento dos modelos foi alterado durante as diferentes gerações.

6. Análise dos resultados

Ao ver os resultados das métricas e das curvas de perda é evidente que os modelos transformers conseguiram desempenhar de forma muito superior às redes LSTM criadas. Para os Ks pequenos, o modelo transformer conseguiu aprender completamente as regras, atingindo perfeição nas métricas para o K = 5 e quase perfeição no K = 1. Enquanto isso, o LSTM pouco progrediu ao longo das épocas, tendo sua loss estabilizada em torno de 0,5. Isso é perceptível no resultado das métricas, atingindo 0,62 de acurácia e 0,72 de F1 score para K = 1, e tendo uma piora no caso de K = 5, com a acurácia de 0.59 e o F1 score de 0.63.





Curvas de perda para os casos de K = 1 e K = 5

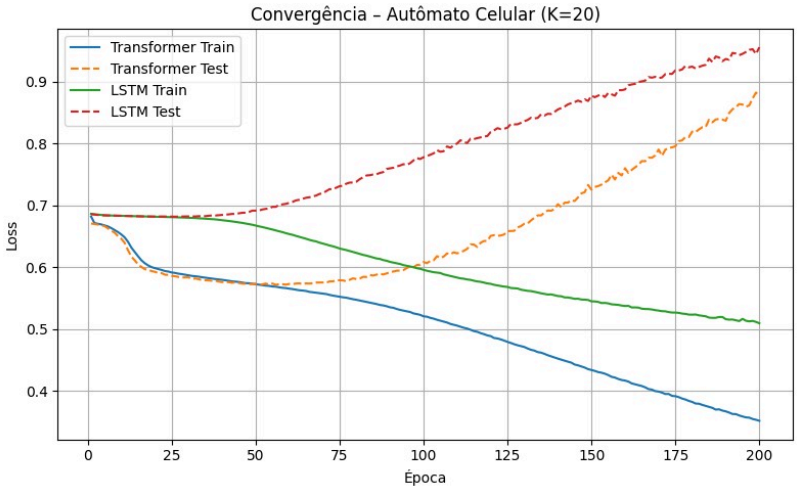
Resultados para K = 1

K = 1	LSTM	Transformer
Acurácia	0.6255	0.9940
Hamming Loss	0.3745	0.0060
F1 Score	0.7259	0.9952

Resultados para K = 5

K = 5	LSTM	Transformer
Acurácia	0.5979	1.0
Hamming Loss	0.4021	0.0
F1 Score	0.6349	1.0

No último caso, K = 20, nenhum dos modelos conseguiu generalizar adequadamente para o problema. Na verdade, é possível ver um comportamento curioso, a partir de certo ponto, em torno de 50 épocas, os modelos começaram a aumentar a loss dos dados de teste, mesmo que a de treino estivesse diminuindo. O comportamento de aprendizado “confuso” se refletiu nas métricas, onde o transformer teve uma queda considerável em relação aos dois casos iniciais, com a acurácia ficando em 0,62 e o F1 score em 0,66. Assim como antes, o modelo LSTM teve uma queda de desempenho e ficou abaixo do transformer.



Curvas de perda para K = 20

Resultados para K = 20

K = 20	LSTM	Transformer
Acurácia	0.5225	0.6296
Hamming Loss	0.4775	0.3704
F1 Score	0.5985	0.6669

7. Referências

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). **Attention is all you need**. Advances in Neural Information Processing Systems (NeurIPS), 30.

Hochreiter, S., & Schmidhuber, J. (1997). **Long short-term memory**. Neural Computation, 9(8), 1735–1780.

Jacot, A., Gabriel, F., & Hongler, C. (2018). **Neural tangent kernel: Convergence and generalization in neural networks**. Advances in Neural Information Processing Systems (NeurIPS), 31.

Belkin, M., Hsu, D., Ma, S., & Mandal, S. (2019). **Reconciling modern machine-learning practice and the classical bias–variance trade-off**. Proceedings of the National Academy of Sciences, 116(32), 15849–15854.